In [ ]:
```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import display

RANDOM_STATE = 42
sns.set_theme(style="whitegrid")
plt.rcParams.update({"figure.dpi": 120})

DATA_DIR = Path("../data")
if not DATA_DIR.exists():
    DATA_DIR = Path("Durga/Lab/MLDL/data")

pd.set_option("display.max_columns", 120)


import gzip
import struct

def load_mnist_images(path, limit=None):
    with gzip.open(path, "rb") as fh:
        magic, count, rows, cols = struct.unpack(">IIII", fh.read(16))
        if magic != 2051:
            raise ValueError(f"Unexpected image magic number: {magic}")
        if limit is None:
            limit = count
        data = np.frombuffer(fh.read(rows * cols * limit), dtype=np.uint8
    return data.reshape(limit, rows, cols).astype("float32") / 255.0

def load_mnist_labels(path, limit=None):
    with gzip.open(path, "rb") as fh:
        magic, count = struct.unpack(">II", fh.read(8))
        if magic != 2049:
            raise ValueError(f"Unexpected label magic number: {magic}")
        if limit is None:
            limit = count
        return np.frombuffer(fh.read(limit), dtype=np.uint8)
```

In [2]:
```python
import tensorflow as tf
from sklearn.metrics import ConfusionMatrixDisplay, classification_report

tf.random.set_seed(RANDOM_STATE)

train_images = load_mnist_images(DATA_DIR / "mnist/train-images-idx3-ubyt
train_labels = load_mnist_labels(DATA_DIR / "mnist/train-labels-idx1-ubyt
test_images = load_mnist_images(DATA_DIR / "mnist/t10k-images-idx3-ubyte.
test_labels = load_mnist_labels(DATA_DIR / "mnist/t10k-labels-idx1-ubyte.
print("Sequence shape:", train_images.shape)
```

```
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1776648555.873324   37493 port.cc:153] oneDNN custom operation
s are on. You may see slightly different numerical results due to floating
-point round-off errors from different computation orders. To turn them of
f, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1776648555.873644   37493 cudart_stub.cc:31] Could not find cu
da drivers on your machine, GPU will not be used.
I0000 00:00:1776648555.905704   37493 cpu_feature_guard.cc:227] This Tenso
rFlow binary is optimized to use available CPU instructions in performance
-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operatio
ns, rebuild TensorFlow with the appropriate compiler flags.
Sequence shape: (8000, 28, 28)
WARNING: All log messages before absl::InitializeLog() is called are writt
en to STDERR
I0000 00:00:1776648556.578953   37493 port.cc:153] oneDNN custom operation
s are on. You may see slightly different numerical results due to floating
-point round-off errors from different computation orders. To turn them of
f, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1776648556.579209   37493 cudart_stub.cc:31] Could not find cu
da drivers on your machine, GPU will not be used.
```

In [3]:

```python
model = tf.keras.Sequential(
    [
        tf.keras.layers.Input(shape=(28, 28)),
        tf.keras.layers.LSTM(64),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(10, activation="softmax"),
    ]
)
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", m
history = model.fit(
    train_images,
    train_labels,
    validation_split=0.15,
    epochs=3,
    batch_size=128,
    verbose=1,
)
```

```
Epoch 1/3
E0000 00:00:1776648557.373201   37493 cuda_platform.cc:52] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: CUDA_ERROR_NO_DEVICE:
no CUDA-capable device is detected
 1/54 ━━━━━━━━━━━━━━━━━━━━ 41s 790ms/step - accuracy: 0.1094 - loss: 2.315
3
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
 7/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1208 - loss: 2.3022
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
13/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1332 - loss: 2.2933
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
19/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1422 - loss: 2.2849
```

```
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
25/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1522 - loss: 2.2755
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
31/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1615 - loss: 2.2653
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
37/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1711 - loss: 2.2538
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
43/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1799 - loss: 2.2415
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
50/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.1896 - loss: 2.2263
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
54/54 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.2676 - loss: 2.0956
- val_accuracy: 0.4233 - val_loss: 1.7437
Epoch 2/3
 1/54 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - accuracy: 0.5234 - loss: 1.6051
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
 6/54 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4567 - loss: 1.6662
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
12/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.4542 - loss: 1.6396
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
17/54 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4593 - loss: 1.6143
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
22/54 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4664 - loss: 1.5897
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
26/54 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4723 - loss: 1.5719
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
31/54 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4791 - loss: 1.5507
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
38/54 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4891 - loss: 1.5219
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
44/54 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4974 - loss: 1.4983
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
51/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.5073 - loss: 1.4711
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
54/54 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.5810 - loss: 1.2683
- val_accuracy: 0.6633 - val_loss: 0.9797
Epoch 3/3
 1/54 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - accuracy: 0.7656 - loss: 0.7908
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓✓
 8/54 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7023 - loss: 0.9004
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
14/54 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7064 - loss: 0.8891
```

```
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
20/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7135 - loss: 0.8705
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
26/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7205 - loss: 0.8553
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
33/54 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7276 - loss: 0.8393
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
40/54 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7337 - loss: 0.8248
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
47/54 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7388 - loss: 0.8118
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
53/54 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7426 - loss: 0.8017
✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓✓
✓✓✓✓✓✓✓✓✓✓✓✓
54/54 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.7747 - loss: 0.7179
- val_accuracy: 0.8042 - val_loss: 0.6243
```

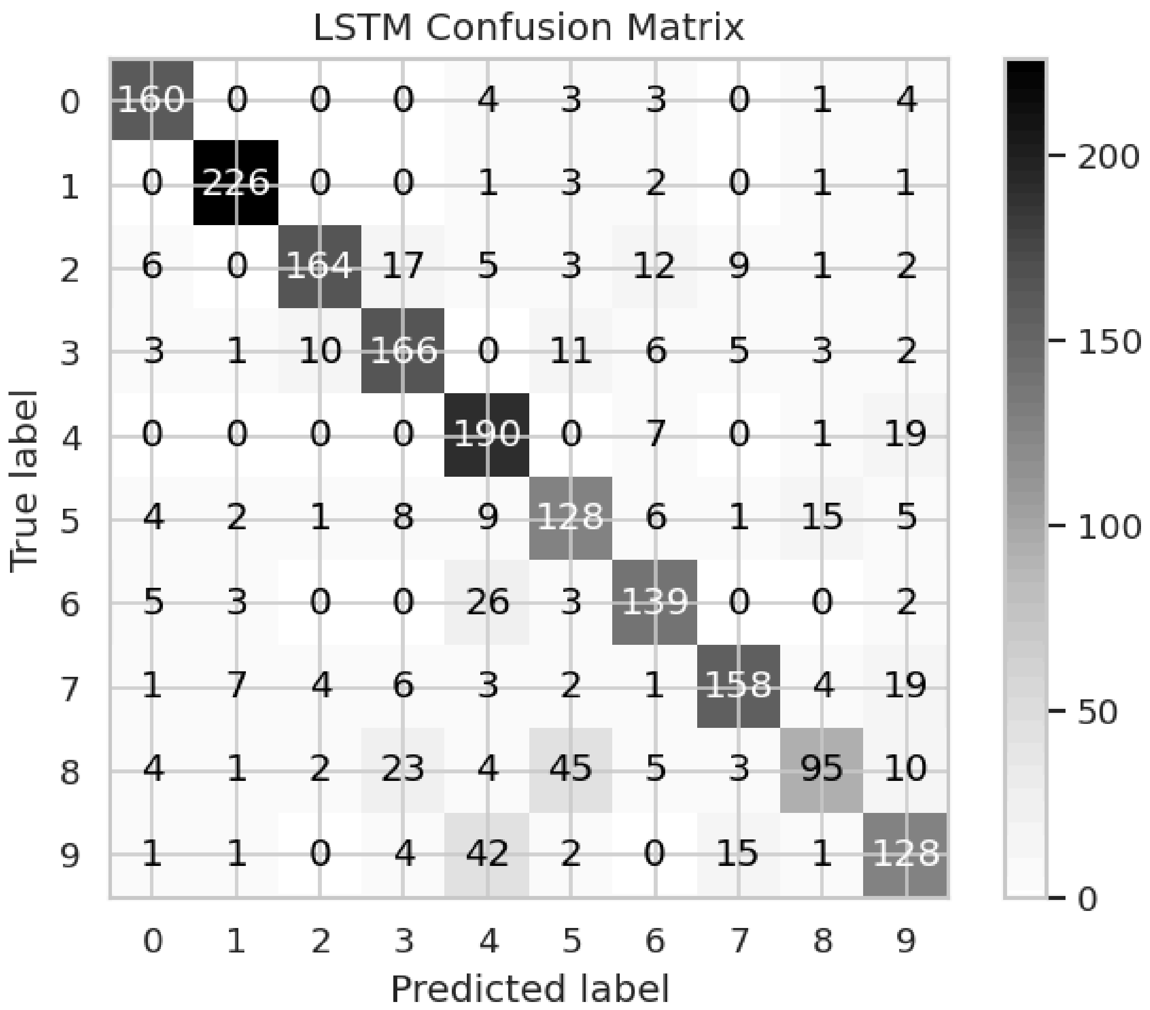
In [4]:

```python
loss, acc = model.evaluate(test_images, test_labels, verbose=0)
print(f"Test accuracy: {acc:.3f}")
pred = model.predict(test_images, verbose=0).argmax(axis=1)
print(classification_report(test_labels, pred))
ConfusionMatrixDisplay.from_predictions(test_labels, pred, cmap="Greys")
plt.title("LSTM Confusion Matrix")
plt.show()
```

```
Test accuracy: 0.777
              precision    recall  f1-score   support

           0       0.87      0.91      0.89       175
           1       0.94      0.97      0.95       234
           2       0.91      0.75      0.82       219
           3       0.74      0.80      0.77       207
           4       0.67      0.88      0.76       217
           5       0.64      0.72      0.68       179
           6       0.77      0.78      0.77       178
           7       0.83      0.77      0.80       205
           8       0.78      0.49      0.61       192
           9       0.67      0.66      0.66       194

    accuracy                           0.78      2000
   macro avg       0.78      0.77      0.77      2000
weighted avg       0.78      0.78      0.77      2000
```


LSTM Confusion Matrix
True label
Predicted label
0
50
100
150
200